

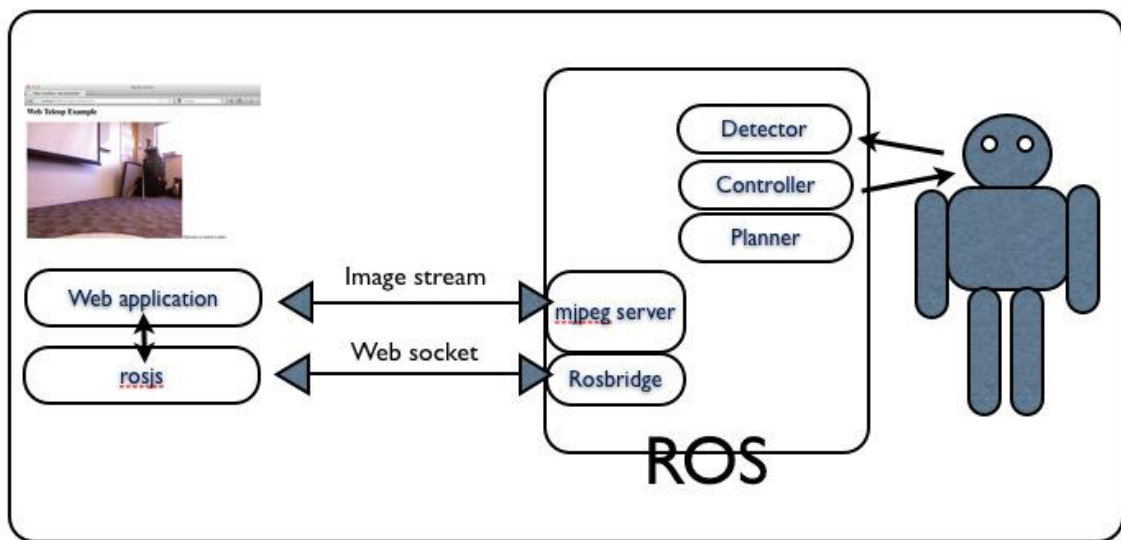


Рисунок 1.1 Архитектура системы rosbridge

Safe, Remote-Access Swarm Robotics Research on the Robotarium (Pickem et al., 2017).

Robotarium — удалённо доступная многоагентная лаборатория для исследований и образования в области роевой робототехники [8]. Платформа состоит из инфраструктуры лаборатории (парк мини-роботов GRITSBot дифференциального привода, глобальное позиционирование верхней камерой по ArUco-меткам, автоматическая зарядка, беспроводное перепрограммирование) и веб-интерфейса с серверным бэкендом. Пользователь через Matlab-API или браузер загружает алгоритм, который сначала проходит симуляцию, затем проверку безопасности и лишь потом исполняется на реальных роботах (Рисунок 1.2). Центральный вклад работы — двухуровневый механизм безопасности: симуляционная верификация (проверка на коллизии и выдача «оценки безопасности» до запуска) и онлайн-барьеры (barrier certificates), при которых пользовательский управляющий сигнал минимально корректируется так, чтобы система оставалась в множестве безопасных состояний — например, при соблюдении минимального расстояния между роботами и границ арены (Рисунок 1.3). Для настоящего проекта значимы проектирование с учётом удалённого доступа, обеспечение безопасности оборудования при исполнении стороннего кода и многопользовательский доступ с очередью; идея барьерных сертификатов прямо перекликается с рассматриваемой далее перспективой серверного контроля рабочей зоны.

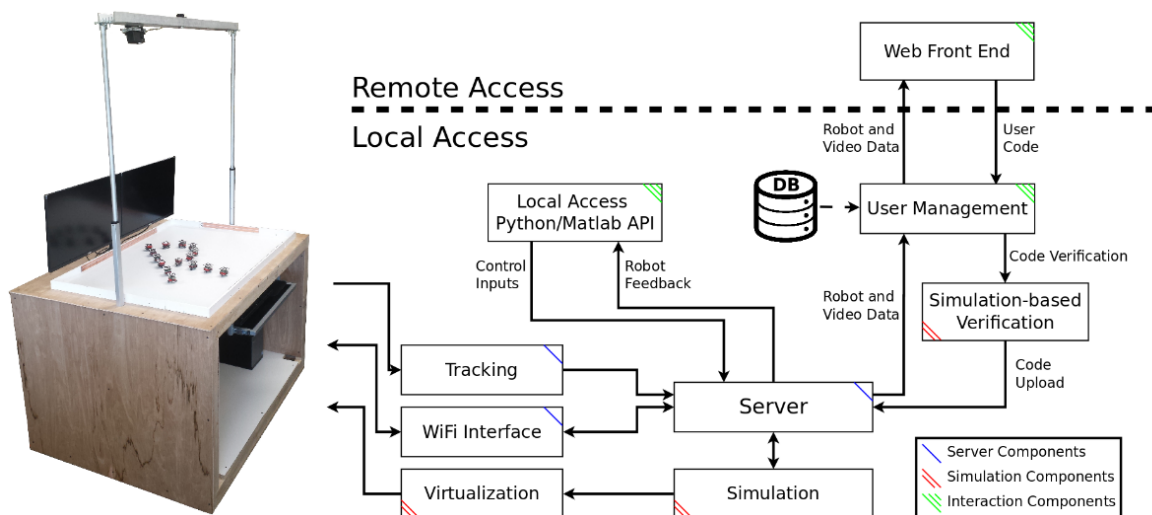


Рисунок 1.2 Архитектура Robotarium

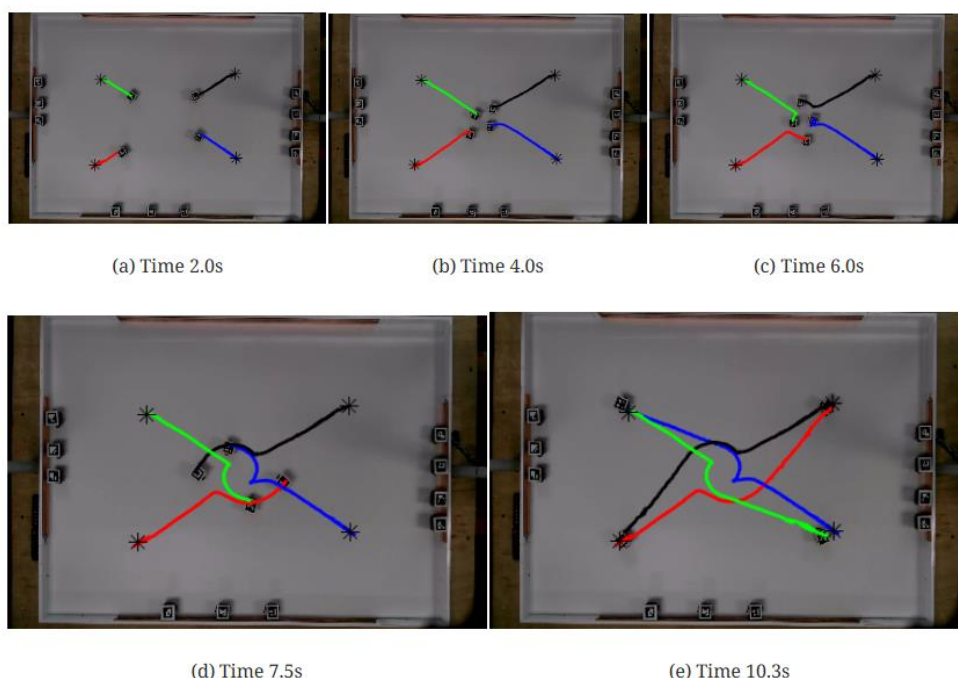


Рисунок 1.3 Демонстрация работы барьеров безопасности

A Remote Lab for Experiments with a Team of Mobile Robots (Casini et al., 2014).

Удалённая лаборатория Университета Сиены для экспериментов с группой мобильных роботов, встроенная в платформу дистанционного обучения АСТ [4]. Стенд — четыре одинаковых робота на базе LEGO Mindstorms NXT в рабочем пространстве 4.5×3 м (Рисунок 1.4); каждый робот описывается кинематической моделью одноколёсника, а на борту выполняется лишь низкоуровневый ПИ-регулятор скорости колёс. Глобальные позиция и ориентация роботов определяются двумя потолочными камерами по маркерам, по завершении эксперимента роботы автоматически приводятся на зарядку. Сервер

реализован как Matlab-функция: на каждом такте она получает позы всех роботов и возвращает желаемые линейную и угловую скорости, связывается с роботами по Bluetooth, передаёт видео и данные в интерфейс и обеспечивает автоматический останов при выходе робота за рабочую зону или сбое связи. Поддерживаются виртуальные препятствия и программно моделируемые сенсоры поверх точных поз от технического зрения, что позволяет испытывать децентрализованные стратегии. Для настоящей работы это решение наиболее показательно сразу по нескольким причинам: безопасность как обязательный модуль, симуляция перед исполнением, программное моделирование сенсоров поверх точной информации о позах и — главное — централизация вычислений на сервере при «тонком» бортовом уровне. Ограничения — привязка к среде Matlab и Java-апплету (снижающая кроссплатформенность) и ориентация на единственный тип роботов.

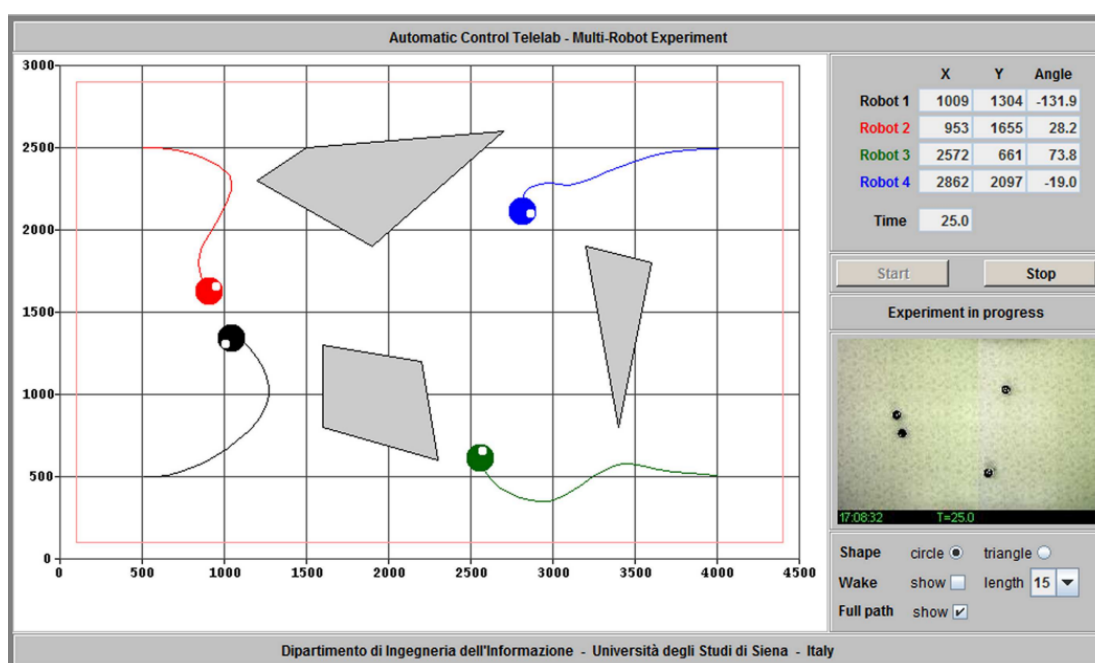


Рисунок 1.4 Пользовательский интерфейс АСТ

хрупкость распределённой ROS-сети поверх нестабильного WiFi. Функционально же большинство роботов — это «простые» машины, которым нужно лишь принимать команды движения и отдавать телеметрию, и для них полноценный бортовой компьютер избыточен.

Ключевое наблюдение, приведшее к итоговой архитектуре, состоит в следующем: роботу не нужны бортовые вычисления как таковые — для дистанционного управления от устройства требуется лишь подключиться к сети и передавать поток байтов между сетью и низкоуровневым контроллером в обе стороны. Поэтому бортовой вычислитель можно полностью убрать, а его функцию «моста» поручить дешёвому Wi-Fi-микроконтроллеру ESP, прошивка которого реализует прозрачный мост «последовательный порт ↔ TCP»: модуль не интерпретирует данные, а лишь пересылает байты между UART и открытым TCP-соединением. Вся «интеллектуальная» часть — ROS, логика команд, очереди, координация — переносится на центральный сервер; с точки зрения серверной ROS-ноды всё выглядит так, будто контроллер подключён к серверу напрямую по кабелю (Рисунок 2.1). Именно отказ от «интеллекта» на борту делает устройство проще, дешевле и надёжнее: вместо хрупкой распределённой ROS-сети — единственный ROS master на сервере и набор простых TCP-туннелей, а добавление новой команды или типа устройства не требует перепрошивки парка. Этот принцип — «тонкое устройство — умный сервер» — и составляет концептуальное ядро итоговой системы. В реализации в роли моста используется модуль ESP-32: по сравнению с ESP8266 он обладает двухъядерным процессором и более стабильным сетевым стеком, сохраняя ту же тривиальную роль «передатчика байтов».

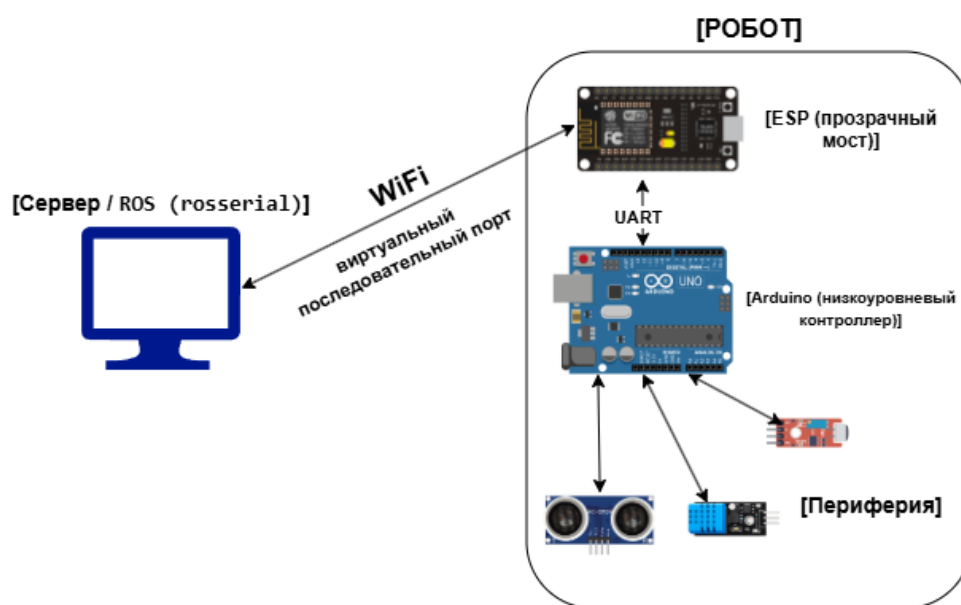


Рисунок 2.1 Схема «ROS+ESP»

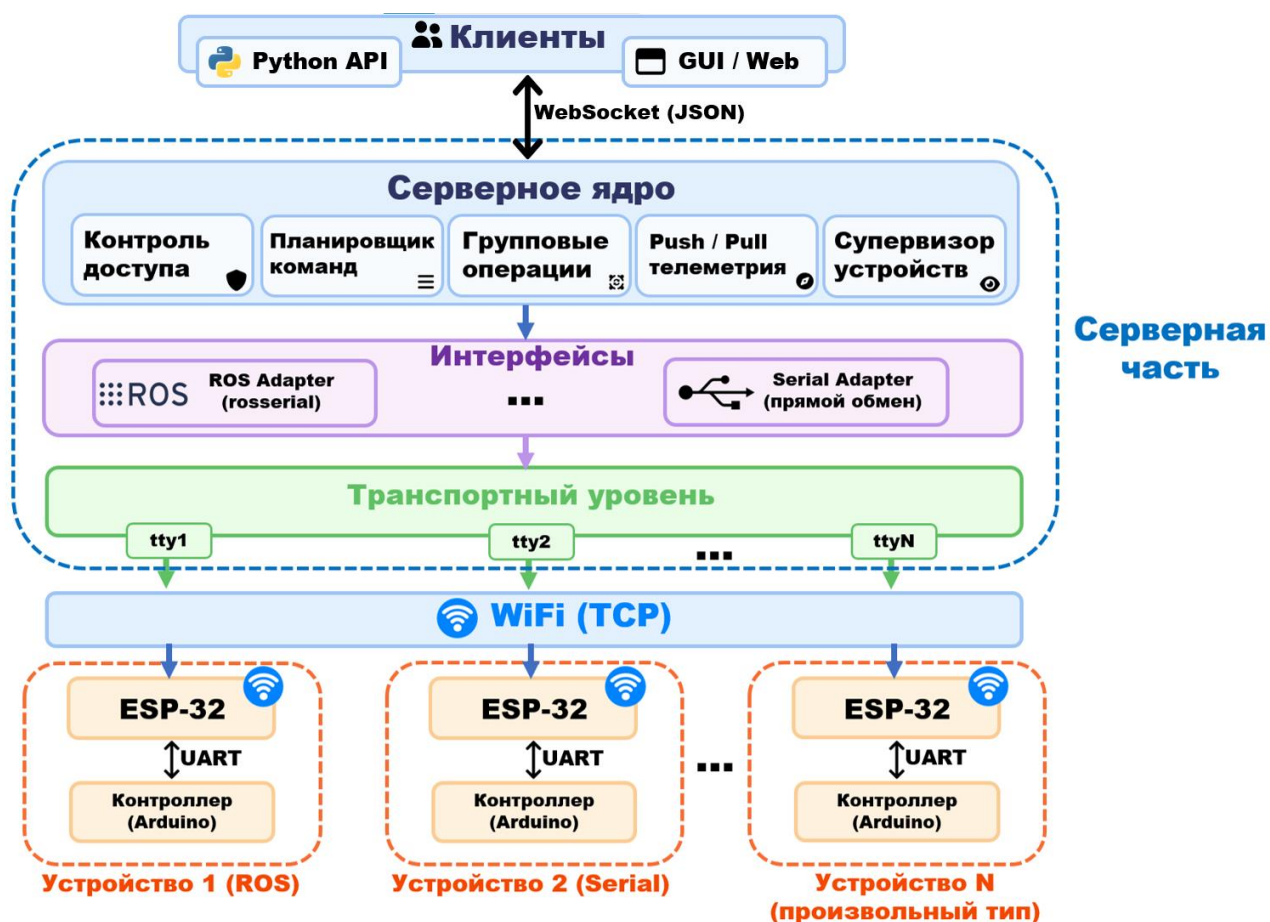


Рисунок 2.2 Архитектурные уровни

Тонкое устройство

На борту робота находятся только два компонента. Сетевой мост ESP-32 реализует прозрачный канал «TCP ↔ UART» [Приложение: Полный код ESP-32 (мост TCP ↔ UART)]: байты, пришедшие из TCP-соединения по WiFi, передаются по UART низкоуровневому контроллеру, и наоборот. Прошивка не интерпретирует содержимое и не меняется при добавлении новых команд или типов устройств. Низкоуровневый контроллер (Arduino) исполняет встроенный клиент roserial: принимает по UART сообщения, преобразует их в непосредственные управляющие воздействия (ШИМ моторов, команды сервоприводам) и публикует обратно показания датчиков. Важно, что контроллер «не знает» о существовании сети — с его точки зрения он соединён с ROS-системой обычным последовательным портом; сеть полностью инкапсулирована в паре «ESP-32 (на роботе) ↔ виртуальный порт socat (на сервере)».

Транспортный конвейер (socat)

Задача транспортного уровня — «вернуть» удалённое устройство на сервер так, будто оно подключено локально по последовательному порту. Эту функцию выполняет утилита socat, которая для каждого устройства создаёт виртуальный последовательный

2.3. Серверное ядро управления

Над уровнем аппаратных интерфейсов располагается ядро системы — оркестратор RemoteLabManager, объединяющий несколько подсистем; их состав и связи показаны на схеме архитектуры сервера (Рисунок 2.3). Все перечисленные подсистемы серверного ядра разработаны и реализованы автором; подробное описание их алгоритмов приводится в Главе 3.

- Драйверы устройств — преобразуют абстрактные команды («двигаться», «сигнал», «повернуть сервопривод») в конкретные воздействия (публикации в топики ROS либо строки в последовательный порт) и отвечают за корректную физическую остановку устройства при прерывании.
- Планировщик команд (CommandScheduler) — ведёт для каждого устройства отдельную очередь с приоритетами и исполняет команды последовательно; поддерживает прерывание текущей команды, что служит основой механизма аварийной остановки.
- Контроль доступа (AccessController) — разграничивает монопольный и общий доступ к устройствам между несколькими клиентами.
- Супервизор устройств (DeviceSupervisor) — отслеживает состояние вспомогательных процессов и перезапускает их при сбоях, приостанавливая на время восстановления соответствующую очередь команд.
- Менеджер групповых процедур (ProcedureManager) — исполняет серверные сценарии над группой устройств.

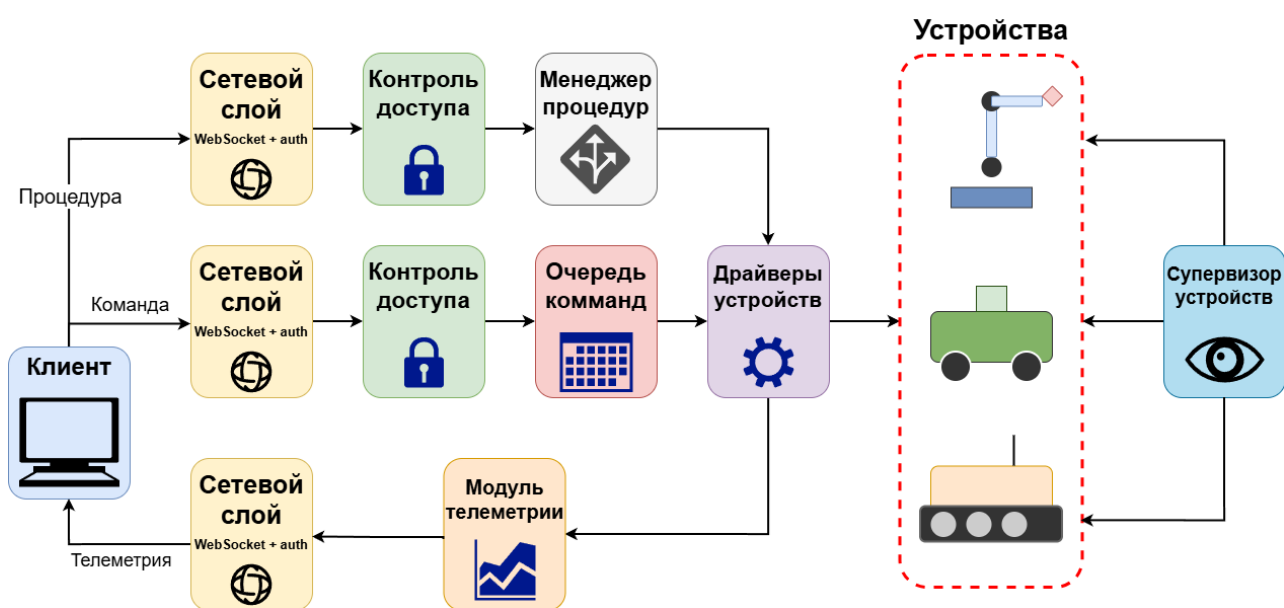


Рисунок 2.1 Архитектура серверного ядра

нескольким устройствам — тогда планировщик помещает её в очередь каждого адресата независимо и отслеживает число ещё не завершивших её устройств, сообщая клиенту о завершении лишь после выполнения всеми; это даёт простой и надёжный механизм синхронных групповых действий поверх независимых очередей.

Обобщённый алгоритм работы рабочего процесса устройства приведён на блок-схеме (Рисунок 3.1), а в компактном виде — в Листинге:

```

цикл worker(устройство):
  команда = очередь.извлечь()           # ожидание по приоритету, при равенстве – FIFO
  дождаться готовности устройства       # пауза на время восстановления супервизором
  если команда.отменена:                # отменена, пока ждала в очереди
    пометить_завершённой(команда); продолжить
  подзадача = запустить(драйвер.исполнить(команда)) # отдельная прерываемая задача
  попытка:
    дождаться(подзадача)
  при отмене:                           # прерывание (E-stop) текущей команды
    зафиксировать_прерывание(команда)
  в любом случае:
    пометить_завершённой(команда)     # учёт многоадресных и уведомление ожидающих
  
```

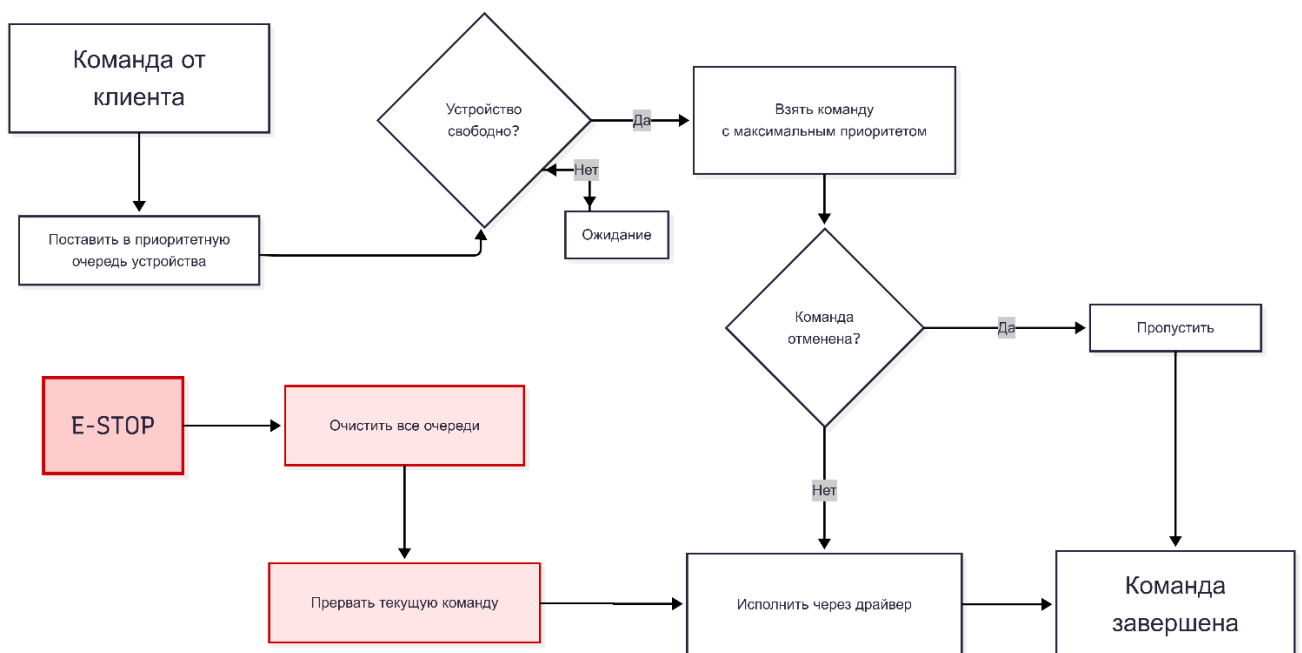


Рисунок 3.1 Блок-схема: очередь команд

Такое разделение «извлечение — ожидание готовности — исполнение как прерываемая подзадача — учёт завершения» позволяет в одном компактном механизме совместить приоритеты, параллелизм по устройствам, аварийное прерывание, паузу на время сбоя и синхронные групповые команды — без явных блокировок и при сохранении строгой последовательности исполнения на каждом устройстве.

понятие логического актуаторного канала: каждая команда относится к одному каналу, и новая команда сбрасывает фоновый «хвост» только своего канала. За счёт этого, например, продолжающий звучать сигнал не прерывается командой движения, и наоборот. Обобщённая схема доставки показана на Рисунке 3.2, а её ядро — в Листинге.



Рисунок 3.2 Схема потоковой доставки

```

исполнить_длительную(целевое_состояние z, длительность T, канал k):
    отменить_хвост(k)                                # сбросить стоп-хвост этого канала
    старт = now()
    пока now() - старт < T:
        опубликовать(z)                                # повтор целевого состояния
        ждать(1 / частота)
    хвост[k] = запустить_фоном( стоп_хвост(k) ) # fire-and-forget, переживает прерывание

стоп_хвост(k):
    старт = now()
    пока now() - старт < T_стоп:
        опубликовать(нулевое_состояние)                # надёжная остановка потоком
        ждать(1 / частота)
    
```

Описанный механизм применяется единообразно ко всем длительным командам: команда движения повторяет потоком целевую скорость, а по завершении — нулевую; команда звукового сигнала повторяет включение, а затем фоновым хвостом — выключение. Тем самым надёжность доставки обеспечивается на уровне драйвера, прозрачно для остальной системы и для пользователя.

Абстракция драйверов (расширение системы)

Решаемая задача. Одно из ключевых требований к лаборатории — унифицированная работа с разнородными устройствами: пользователь должен управлять любым роботом группы через единый интерфейс, не вникая в то, на каком стеке и по какому протоколу работает конкретное устройство. При этом сама группа гетерогенна: в неё могут входить и роботы на базе ROS, и простые устройства с прямым обменом по последовательному порту, и др. Чтобы совместить единый внешний интерфейс с

3.4. Групповые процедуры

Серверные групповые процедуры, концептуально описанные в Главе 2, реализованы подсистемой ProcedureManager. Процедура — это исполняемый на сервере асинхронный сценарий, которому доступен весь интерфейс оркестратора: захват и освобождение устройств, постановка команд с ожиданием их физического выполнения, чтение телеметрии, запуск и отмена. Менеджер процедур ведёт реестр запущенных сценариев, изолирует их по доступу (процедура захватывает устройства наравне с обычным клиентом) и обеспечивает их корректное завершение, в том числе принудительную отмену с гарантированной остановкой задействованных роботов. В составе системы реализованы три процедуры: аварийная остановка всей лаборатории (stop_all), групповой тест синхронизации (sync_test — синхронная подача сигнала и разворот всех роботов с проверкой живой телеметрии каждого) и навигационная процедура «По домам» (all_go_home). Ниже подробно рассмотрена последняя как наиболее показательный пример скоординированного группового поведения; её прикладная логика предварительно отработана автором на отдельном симуляторе навигации [7].

Навигационная процедура «По домам»: модель, алгоритмы, реализация

Процедура приводит каждый робот группы из текущей позиции в назначенную ему домашнюю ячейку, объезжая препятствия, при централизованной координации всей группы сервером. Контур управления одного робота образуют четыре связанных компонента — модель движения, оценка состояния (фильтр), планировщик пути и контроллер движения, — объединённые серверным циклом (Рисунок 3.3). Далее каждый из них описан формально.

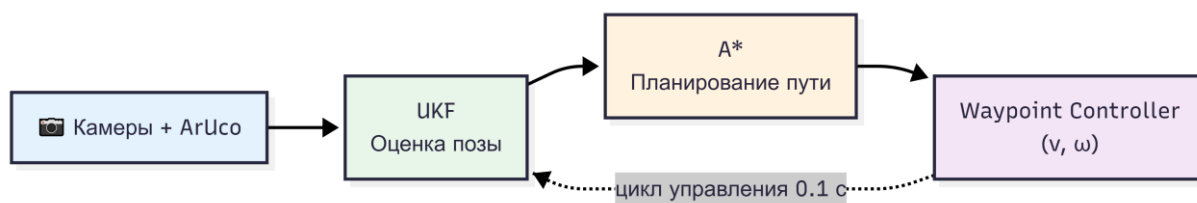


Рисунок 3.3 Концептуальная схема процедуры «По домам»

Модель движения и измерений. Робот описывается кинематической моделью одноколёсника (unicycle) [1] с состоянием $s = [x, y, \theta]$ (координаты и курс) и управлением $u = [v, \omega]$ (линейная и угловая скорости). Дискретная модель движения с шагом Δt :

Глава 4. ЭКСПЕРИМЕНТАЛЬНАЯ ПРОВЕРКА СИСТЕМЫ

Предыдущие главы описали архитектуру системы и её программную реализацию. Настоящая глава посвящена экспериментальной проверке работоспособности и характеристик разработанной лаборатории на реальном оборудовании. Сначала описывается собранный экспериментальный стенд и методика испытаний, затем приводятся результаты количественных измерений (задержка управления, устойчивость к потерям пакетов, реакция на разрывы связи) и качественной проверки групповых сценариев — группового теста синхронизации и навигационной процедуры «По домам». Цель главы — показать, что реализованная система действительно обеспечивает удалённое управление реальным роботом в реальном времени и что заложенные в архитектуру механизмы (надёжная доставка, отказоустойчивость, групповая координация на основе технического зрения) работают на практике.

4.1. Экспериментальный стенд

Для испытаний был собран физический полигон (Рисунок 4.1), воспроизводящий рабочие условия виртуальной лаборатории.

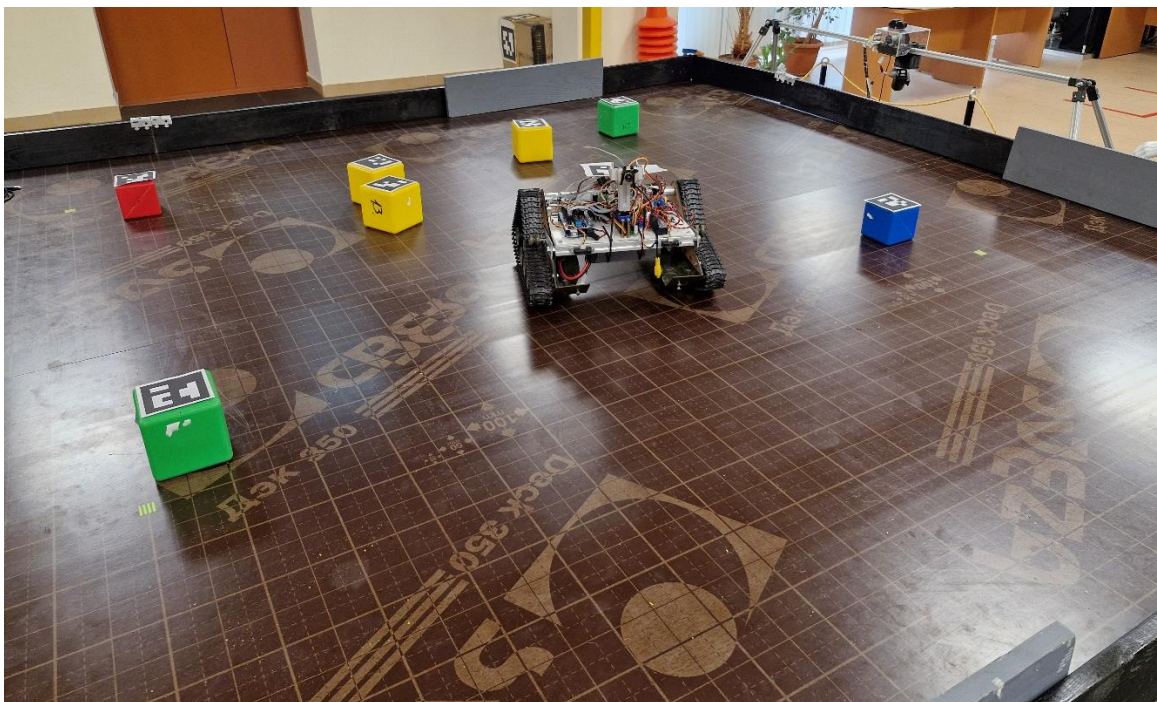


Рисунок 4.1 Экспериментальный стенд

Стенд включает следующие компоненты:

- **Рабочее поле (полигон).** Размеченная горизонтальная площадка с расставленными препятствиями (кубики), ограниченная по периметру; в одной из угловых зон задана



Рисунок 4.1 Экспериментальный стенд в работе

4.2. Количественные и качественные измерения

Количественные измерения характеризуют качество канала и реакцию системы на сбой: средняя задержка «команда → исполнение» и «телеметрия → клиент», доля теряемых кадров на беспроводном участке и время восстановления при разрывах связи

Задержка управления и телеметрии

Задержка — ключевая характеристика для управления в реальном времени. Измерялось время от отправки команды клиентом до её исполнения на роботе и время доставки снимка телеметрии от робота до клиентского callback. В установившемся режиме на лабораторном Wi-Fi средняя сквозная задержка управляющего тракта составила ≈ 20 мс, что для задач телеуправления мобильным роботом субъективно неощутимо и достаточно для замкнутого контура навигационной процедуры, работающей с тактом в десятки миллисекунд. Задержка обратного (телеметрического) тракта оказалась того же порядка. Полученное значение подтверждает пригодность выбранного транспорта (WebSocket поверх постоянного соединения + туннель socat с отключённым алгоритмом Нейгла) для управления в реальном времени.

Устойчивость к потерям пакетов

Беспроводной участок и протокол rosserial поверх виртуального порта подвержены эпизодическим потерям кадров. На стенде доля потерь на беспроводном участке в штатном режиме не превышала $\approx 5\%$. Несмотря на это, управление оставалось

4.3. Навигационная процедура «По домам» для роботов uarp-13

Центральный сценарный эксперимент — запуск навигационной процедуры «По домам» (AllGoHome) на собранном полигоне с двумя камерами. В эксперименте участвовал **два робота архитектуры uarp-13**: на рабочем поле были расставлены препятствия и заданы домашние точки, после чего на сервере была запущена процедура. В ходе исполнения сервер централизованно выполнял весь навигационный контур, описанный в Главе 3: по измерениям **двух камер** (ArUco-маркеры роботов) сигма-точечный фильтр Калмана (UKF) давал устойчивую оценку поз; по дискретной карте полигона с инфляцией препятствий алгоритм A* строил маршруты от текущей ячейки роботов до домашних; waypoint-контроллер вёл роботов по маршрутам, а управляющие воздействия выдавались роботу через штатный конвейер команд лаборатории — теми же драйверами, что и обычные пользовательские команды.

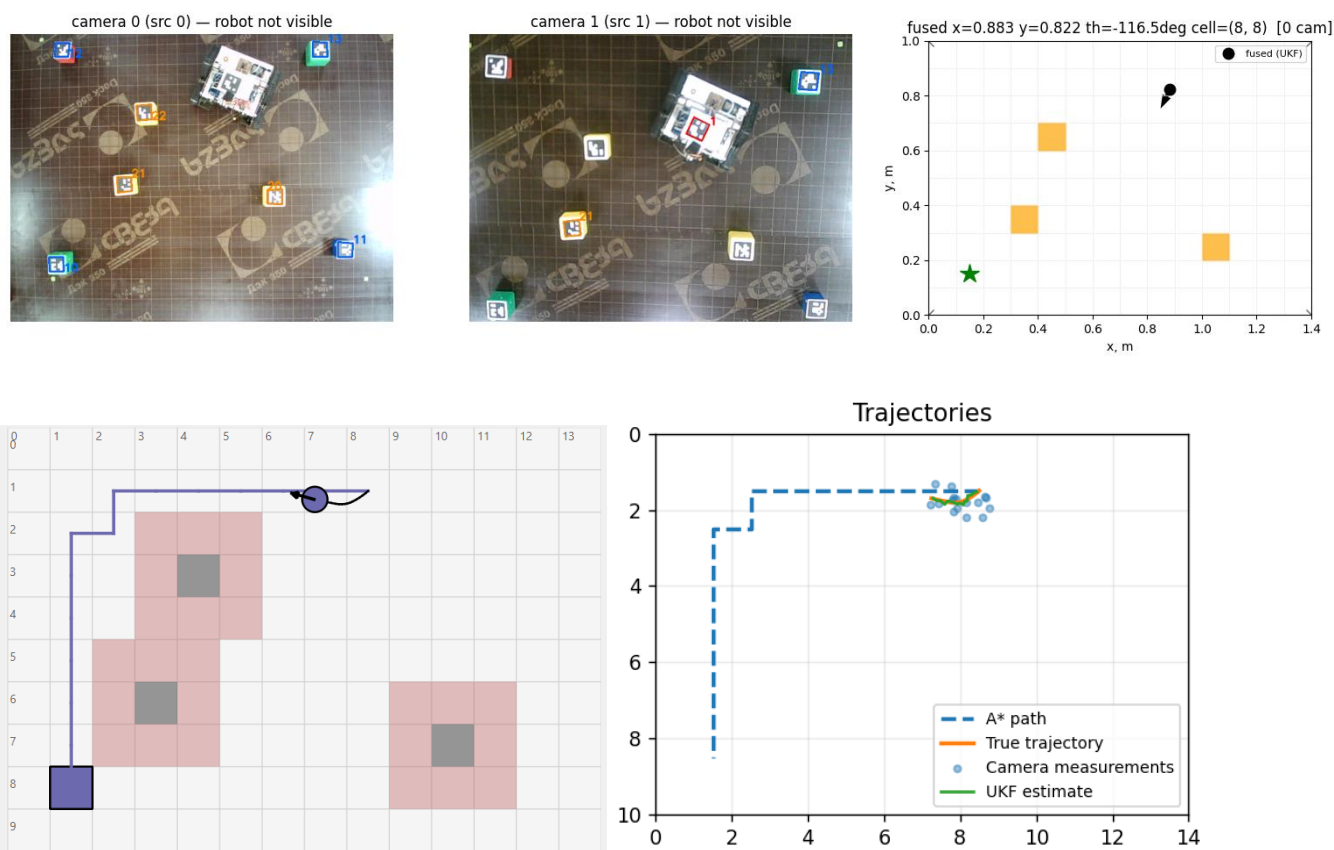


Рисунок 4.2 Тест процедуры «по домам»

Роботы успешно прокладывали маршрут в обход препятствий и доезжали до назначенной домашней точки; на всём протяжении движения оценка позы (по двум камерам) и план маршрута отображались на серверной реконструкции сцены (Рисунок 4.2). Эксперимент подтверждает работоспособность всей вертикали групповой навигации на реальном оборудовании: централизованное восприятие (две камеры + ArUco),